

CS772: PROBABILISTIC MACHINE LEARNING

ASSIGNMENT - 2

- VEDHANTH BALASUBRAMANIAN
(231135)

PART 1

Q1.1 Reconstruction Loss (10pts)

$$1. p_{\theta}(x|z) = \mathcal{N}(x|f_{\theta}(z), \sigma^2 I) = \frac{1}{(2\pi\sigma^2)^{D/2}} \exp\left(-\frac{1}{2\sigma^2} \|x - f_{\theta}(z)\|^2\right)$$

$$\log p_{\theta}(x|z) = -\frac{D}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|x - f_{\theta}(z)\|^2$$

Since we are minimizing w.r.t θ , we can ignore $-\frac{D}{2} \log(2\pi\sigma^2)$

$$\Rightarrow \text{minimizing } -\frac{1}{2\sigma^2} \|x - f_{\theta}(z)\|^2$$

$$\Rightarrow \text{minimizing } \|x - f_{\theta}(z)\|^2$$

which is exactly the Mean Squared error function
Hence Proved.

2. When the network only outputs the mean $f_{\theta}(z)$ and assumes a fixed variance, the model fails to capture the variance of the data distribution. Since the model attempts to minimize MSE, it tends to output the geometric average of plausible images rather than a single distinct sharp image. With a learned variance on the other hand, the model can confidently predict a sharp distinct image while capturing its uncertainty through the learned variance.

3. Modelling each pixel as an independent Bernoulli random variable corresponds to the Binary Cross Entropy loss function. Raw image pixel intensities range from 0 to 255, but when they are scaled to the interval $[0, 1]$, these normalized values can be directly interpreted as the probability of a pixel being on (white) versus off (black). A Bernoulli distribution aligns with this providing a better bound than the Gaussian distribution which assumes values can range from $-\infty$ to ∞ .

Q1.2 Closed-Form KL Divergence

Ans. $q_\phi(z|\mu) = \mathcal{N}(z|\mu, \Sigma)$, $\Sigma = \text{diag}(\sigma^2)$

$p(z) = \mathcal{N}(z|0, I)$

$$KL[q_\phi(z|\mu) \| p(z)] = E_{q_\phi}[\log q_\phi(z|\mu) - \log p(z)]$$

$$q_\phi(z|\mu) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(z-\mu)^T \Sigma^{-1}(z-\mu)\right)$$

$$\log q_\phi(z|\mu) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2}(z-\mu)^T \Sigma^{-1}(z-\mu)$$

$|\Sigma| = \prod_{i=1}^d \sigma_i^2$ since $\Sigma = \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_d^2)$

$$\Rightarrow \log |\Sigma| = \sum_{i=1}^d \log \sigma_i^2$$

Σ^{-1} is just the diagonal matrix of inverted variances.

$$(z-\mu)^T \Sigma^{-1}(z-\mu) = \sum_{i=1}^d \frac{(z_i - \mu_i)^2}{\sigma_i^2}$$

$$\Rightarrow \log q_\phi(z|\mu) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^d (\log \sigma_i^2) - \frac{1}{2} \sum_{i=1}^d \frac{(z_i - \mu_i)^2}{\sigma_i^2}$$

$$\log p(z) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^d z_i^2$$

$$\Rightarrow KL[q_\phi(z|\mu) \| p(z)] = E_{q_\phi} \left[-\frac{1}{2} \sum_{i=1}^d \left(\log \sigma_i^2 + \frac{(z_i - \mu_i)^2}{\sigma_i^2} - z_i^2 \right) \right]$$

By linearity of expectation, the expectation goes inside the summation. Also,

$$E[z_i] = \mu_i, \quad E[(z_i - \mu_i)^2] = \sigma_i^2 = (\text{Var } z_i)$$

$$\text{Var}(z_i) = E[z_i^2] - \mu_i^2$$

$$\Rightarrow E[z_i^2] = \sigma_i^2 + \mu_i^2$$

$$\Rightarrow KL[q_\phi(z|\mu) \| p(z)] = -\frac{1}{2} \sum_{i=1}^d \log \sigma_i^2 + \frac{1}{2} \sum_{i=1}^d (\mu_i^2 - \sigma_i^2) \quad \text{Hence Proved}$$

PART 2 Implementation

Q2.1 Reparametrization

Ans: To train the VAE using back propagation, we optimize the encoder parameters ϕ by maximizing the ELBO.

$$\mathcal{L}(\theta, \phi) = E_{z \sim q_{\phi}(z|x)} [\log(p_{\theta}(x|z))] - KL[q_{\phi}(z|x) || p(z)]$$

The difficulty arises in the reconstruction term, (The KL loss can be computed in closed form). The expectation is taken over a distribution which depends on ϕ , making it difficult to ~~push~~ compute the gradient. The reparametrization trick by rewriting z as $z = \mu_{\phi} + \sigma_{\phi} \odot \epsilon$ $\epsilon \sim N(\epsilon|0, I)$

Now the randomness is isolated in ϵ which doesn't depend on ϕ . Now, the gradient can be easily computed by pushing the gradient operator inside the expectation.

$$\nabla_{\phi} E_{\epsilon \sim N(0, I)} [\log p_{\theta}(x|z)] = E_{\epsilon \sim N(0, I)} [\nabla_{\phi} \log p_{\theta}(x|\overset{\uparrow}{z})]$$

This makes the sampling step differentiable w.r.t μ_{ϕ} and σ_{ϕ} allowing standard back propagation to update the encoder.

Q2.4 Monte Carlo KL Approximation

Ans. As L increases, the Monte Carlo estimate of the KL term becomes less noisy because it averages over more latent samples. The $L=1$ curve is typically more variable and can deviate from the closed form baseline, while $L=100$ is smoother and usually closer to the exact KL term. In our case, the difference is visible but not dramatic.

Note: I also observed that by changing the seed values, the relative positions of the $L=1, 10, 100$ curves changed w.r.t the baseline solution, probably something to do with the random initializations of the model in the start.

If the variance is high, gradients are noisy which gives the encoder a less reliable signal. This is problematic in early training when the latent space is still poorly organised and unstable updates can slow convergence or push the model in inconsistent directions.

When the KL divergence has a closed form expression, it is gives an exact and deterministic regularization term with no sampling noise. In contrast, Monte Carlo estimation introduces variance into the objective and into the gradient, so even large L produce run to run differences due to randomness in initialization, minibatch order, latent sampling.

Part 3 β VAE and Posterior collapse

Q3.2 Posterior Collapse Analysis.

Ans. When β is large (like when $= 10$), most of the latent dimensions undergo posterior collapse, that is the encoder output becomes independent of x and the posterior $q_\phi(z|x)$ moves towards $p(z) = \mathcal{N}(z|0, I)$. In this case

This happens because a large β places a strong penalty on the KL divergence term in the ELBO which results in the encoder taking the easiest path, that is produce the same posterior for ~~every input~~ most of the inputs. This is why large β ~~gives~~ incentivizes the encoder to ignore the input.

Hence, with high β , the decoder is essentially being fed pure Gaussian noise that is completely different from the image. Without any distinct features of the original image, the decoder receives no useful signals.

Note: The variance plotted in the graph in the previous parts represent the variance of the encoder means. ~~As~~ With $\beta = 10$, this approaches 0, indicating the model gives the exact same mean vector regardless of the image. On the other hand, the variance of the approx posterior distribution (σ^2 of $q(z|x)$) approaches 1 to match the prior.

Part 4 Geometry of latent space

84.1 Linear Interpolation

Ans 4. Yes, the figure shows a fairly smooth semantic transition from 2 to 7. At $t=0.0$ to $t=0.3$, the decoded digits look like a 2, but they slowly become thinner. From $t=0.4$ to $t=0.6$ the image becomes ambiguous, it starts losing the 2 structure and picks up an angled stroke which makes it look like a 7. At $t=0.7$ onward the output is clearly a 7. So, the interpolation does not jump abruptly; it morphs continuously.

This happens because the KL term pushes the encoder to follow a standard normal prior which encourages the latent space to be organised and continuous. That means nearby latent points tend to decode to similar looking digits, so linear interpolation stays on a meaningful path.

A standard autoencoder term without a KL term, makes the latent codes scattered arbitrarily and the decoder only needs to work on those exact encoded points. So, the straight line connecting two codes may pass through regions the decoder never learned to handle which can produce weird or discontinuous outputs.

Note: The starting 2 in my code is not perfectly clean which makes some of the intermediate images look like ~~an 8~~ like the lower portion of an 8.

34.2 2D Latent Space Visualization

Ans. The 2D latent space shows meaningful structure grouping many samples by digit class, but the groups aren't fully separated. Several digits form recognisable clusters which don't overlap much with other clusters, eg. 1, 7, 0. Visually similar digits tend to lie near one another eg. 4, 9 and 3, 5, 8; also showing a good amount of overlap.